profiling ( ) : ... rall

=> helps ...

=> ber...

lee...

... copies / faster

... GPU ... memory

parallelism primitives

3 important ideas

... + all ... gather

... 5

po... parallel

Tensor parallel

③: Activation parallelism : manage footprint of memory

sequence parallel => handle long activation memory for long sequence / large Batch size / large model.

=> Naive Data parallelism : split the elements of B sized batch across M machines. Exchange gradients to synchronize

⇓

fit Batch of params to various GPUs

⇓ [fusion solution] => launch kernel, memo allocation, synchronization, ⇐ [extra time / work spent on managing → not useful actual computation]

memory issues : we need 5 copies of weights and 16 Bytes per param

⇓

ZeRO: Solving the memory over head (开销) issue of DP

core idea: split up the expensive parts (state) and use the reduce-scatter equivalence

zero stage 1: optimizer state sharding

=> split up the optimizer state (first + second moments) across GPUs
everyone has the parameters + gradients

Each worker is responsible for updating a subset of params (corresponding to their slice)

{ gradients, optimizer states } => update parameters => optimization

Zero stage #1: How it works? { sync gradient (adam/adamw/SGD), shard optimizer states across model, same compute/communication cost

step 1: everyone compute full gradient on their subset of the batch

step 2: ReduceScatter the gradients - incur # params communication cost

step 3: Each machine updates their param using their gradient + optimizer state

step 4: All gather the parameters - incur # params communication cost

---

Broadcast: One GPU (rank) sends same message or data to all other GPUs

Reduce: combine values from all GPUs (sum, max, etc) and keep the result on one rank. (to GPU)

All-reduce: combine values from all ranks, the distribute the total result back to every GPU (rank)

Scatter: one GPU splits a dataset and sends a different slice to each GPU